

Anexo 01B — Herencia, Constructores y Métodos en C#

Asignatura: Programación II | Instituto de Educación Superior N.º 9007 Dr. Salvador Calafat | Ciclo Lectivo 2026

Herencia en C#

La **herencia** es el mecanismo por el cual una clase (*clase derivada* o *subclase*) adquiere los miembros (propiedades, métodos) de otra clase (*clase base* o *superclase*). La clase derivada puede usar los miembros heredados, sobrescribirlos o agregar nuevos.

En C# la herencia se declara con `:` después del nombre de la clase. Solo se puede heredar de **una única clase base** (herencia simple), pero se pueden implementar múltiples interfaces.

```
1 // clase base – define comportamiento compartido por todas las subclases
2 public class Vehicle
3 {
4     public string Brand { get; set; }
5     public void Start() => Console.WriteLine("Motor encendido");
6 }
7
8 // clase derivada – hereda de Vehicle con ":"
9 public class Car : Vehicle
10 {
11     // propiedad propia – no estaba en Vehicle
12     public int Doors { get; set; }
13 }
```

```
1 // Car tiene Brand y Start() de Vehicle, más Doors propia
2 Car car = new Car();
3 car.Brand = "Toyota"; // heredado de Vehicle
4 car.Start();           // heredado de Vehicle
5 car.Doors = 4;         // propio de Car
```

Modificadores clave en herencia

Modificador	Dónde se usa	Efecto
<code>abstract</code>	Clase / método de la base	La clase no puede instanciarse. El método debe ser implementado por la subclase.

Modificador	Dónde se usa	Efecto
<code>virtual</code>	Método de la base	La subclase puede sobrescribirlo con <code>override</code> . Si no lo hace, usa la implementación base.
<code>override</code>	Método de la subclase	Sobrescribe un método <code>virtual</code> o <code>abstract</code> de la clase base.
<code>sealed</code>	Clase / método	En clase: no puede heredarse. En método: no puede sobrescribirse más.
<code>protected</code>	Miembro de la base	Accesible solo desde la clase base y sus subclases — no desde código externo.

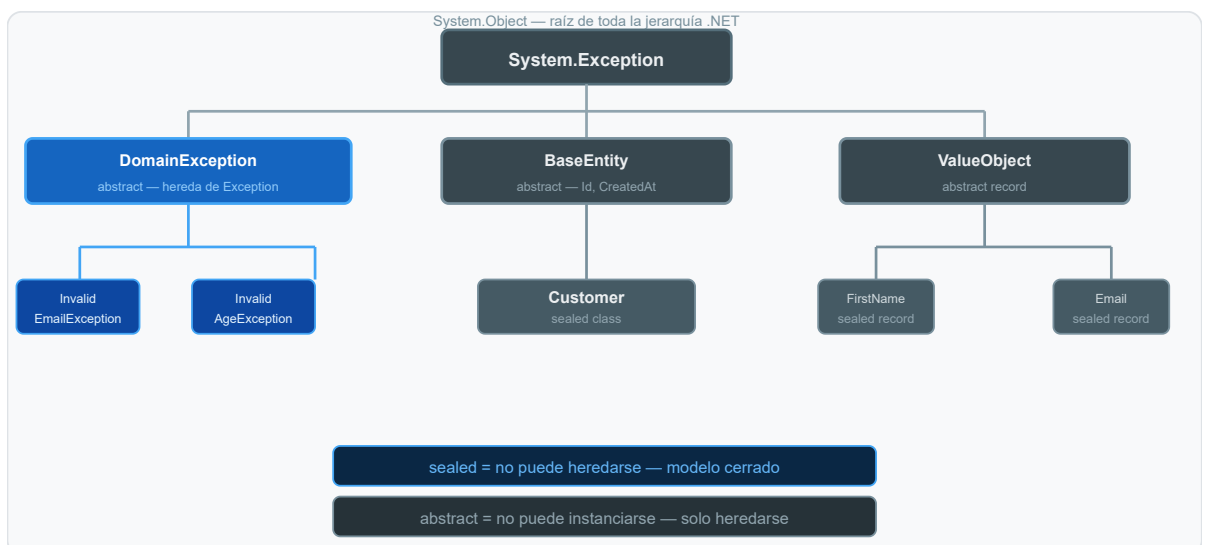
Sobrescritura de métodos con `virtual` y `override`

```

1 public class Vehicle
2 {
3     // virtual: la subclase puede (pero no está obligada) sobrescribirlo
4     public virtual void Describe()
5         => Console.WriteLine("Soy un vehiculo");
6 }
7
8 public class Car : Vehicle
9 {
10    // override: reemplaza la implementación de Vehicle
11    public override void Describe()
12        => Console.WriteLine("Soy un auto");
13 }

```

Herencia en el proyecto CommerX



Constructores en C#

Un **constructor** es un método especial que se ejecuta automáticamente cuando se crea una instancia con `new`. Su responsabilidad es dejar el objeto en un estado válido desde el primer momento. Tiene el mismo nombre que la clase y no declara tipo de retorno.

Constructor sin parámetros

```
1 public class Order
2 {
3     public Guid Id { get; }
4     public DateTime CreatedAt { get; }
5
6     // se ejecuta automáticamente al hacer "new Order()"
7     public Order()
8     {
9         // estado inicial garantizado desde el momento de creación
10        Id = Guid.NewGuid();
11        CreatedAt = DateTime.UtcNow;
12    }
13 }
```

Constructor con parámetros

```
1 public class Product
2 {
3     public string Name { get; }
4     public decimal Price { get; }
5
6     // recibe los datos necesarios para construir un Product válido
7     public Product(string name, decimal price)
8     {
9         // los parámetros se asignan a las propiedades del objeto
10        Name = name;
11        Price = price;
12    }
13 }
```

Encadenamiento con `: base()`

Cuando una clase hereda de otra y la clase base tiene un constructor con parámetros, la subclase debe llamarlo explícitamente con `: base(...)`. Esta llamada ocurre *antes* de ejecutar el cuerpo del constructor de la subclase.

```
1 public abstract class Animal
2 {
3     public string Name { get; }
4
5     // protected: solo accesible desde las subclases
6     protected Animal(string name)
7     {
```

```

8      Name = name;
9    }
10 }
11
12 public class Dog : Animal
13 {
14     public string Breed { get; }
15
16     // ": base(name)" llama al constructor de Animal primero
17     public Dog(string name, string breed) : base(name)
18     {
19         // en este punto Name ya fue asignado por Animal
20         Breed = breed;
21     }
22 }

```

Cadena de constructores en CommerX:

InvalidEmailException(string value)

llama a → DomainException(string message)

llama a → Exception(string message)

Cada nivel inicializa lo suyo y delega el resto hacia arriba con `: base(...)`.

Constructor privado y método de fábrica Create()

Cuando un constructor es `private`, nadie fuera de la clase puede crear instancias con `new` directamente. El único camino es usar un método estático que actúa como puerta de entrada controlada. Este es el patrón que usan todos los Value Objects del proyecto.

```

1 public sealed record Temperature : ValueObject
2 {
3     public double Celsius { get; }
4
5     // private: nadie puede hacer "new Temperature()" desde fuera
6     private Temperature(double celsius)
7     {
8         Celsius = celsius;
9     }
10
11     // static: se llama sobre el tipo, no sobre una instancia
12     // único punto de entrada – valida antes de construir
13     public static Temperature Create(double celsius)
14     {
15         // Fail Fast: valor físicamente imposible
16         if (celsius < -273.15)
17             throw new ArgumentException("Por debajo del cero absoluto.");
18
19         // solo llega aquí si el valor es válido
20         return new Temperature(celsius);
21     }

```

22 }

Métodos en C#

Un **método** es un bloque de código con nombre que realiza una operación. Puede recibir datos (parámetros), ejecutar lógica y opcionalmente devolver un resultado.

Anatomía de un método

```

1 // [modificador de acceso] [static?] [tipo de retorno] [nombre]([parámetros])
2 public static int Add(int a, int b)
3 {
4     // cuerpo - lógica del método
5     return a + b;
6 }
```

Tipos de retorno

Retorno	Significado	Ejemplo
<code>void</code>	No devuelve ningún valor	<code>void Print(string text)</code>
Tipo concreto	Devuelve un valor del tipo indicado	<code>int Add(int a, int b)</code>
<code>Task</code>	Operación asincrónica sin valor de retorno	<code>async Task SaveAsync(Customer c)</code>
<code>Task<T></code>	Operación asincrónica que devuelve un valor	<code>async Task<bool> ExistsAsync(string doc)</code>

Modificadores de acceso

Modificador	Accesible desde	Uso típico en CommerX
<code>public</code>	Cualquier código	Métodos de API pública: <code>Create()</code> , <code>Execute()</code>
<code>private</code>	Solo la misma clase	Constructores de VOs, métodos auxiliares internos
<code>protected</code>	La clase y sus subclases	Constructor de <code>DomainException</code> , <code>BaseEntity</code>
<code>internal</code>	El mismo proyecto	Implementaciones de Infrastructure que no se exponen

Métodos estáticos vs métodos de instancia

Un método **de instancia** opera sobre los datos de un objeto concreto y se llama sobre una variable que contiene la instancia. Un método **estático** (`static`) pertenece a la

clase, no a ninguna instancia — se llama directamente sobre el tipo sin necesidad de `new`.

```
1 public class Calculator
2 {
3     public int LastResult { get; private set; }
4
5     // instancia: accede a LastResult — necesita un objeto creado con new
6     public int Add(int a, int b)
7     {
8         LastResult = a + b;
9         return LastResult;
10    }
11
12    // estático: no accede a LastResult — no depende de ninguna instancia
13    public static int Multiply(int a, int b) => a * b;
14 }
```

```
1 // método de instancia — requiere crear el objeto primero
2 Calculator calc = new Calculator();
3 int sum = calc.Add(3, 4);
4
5 // método estático — se llama directamente sobre el tipo, sin instancia
6 int product = Calculator.Multiply(3, 4);
```

En CommerX:

`Email.Create("x@y.com")` — método **estático**: se llama sobre el tipo, sin instancia previa.

`customer.Equals(other)` — método **de instancia**: se llama sobre un objeto `Customer` existente.

Métodos asincrónicos con `async / await`

Cuando un método realiza operaciones de entrada/salida (acceso a base de datos, red, archivos), se declara `async` y retorna `Task` o `Task<T>`. Esto evita bloquear el hilo de ejecución mientras se espera la respuesta. En el proyecto, todos los métodos del repositorio son asincrónicos.

```
1 // async: el método puede contener await — no bloquea el hilo
2 // Task: retorna una operación asincrónica sin valor de retorno
3 public async Task AddAsync(Customer customer)
4 {
5     // await: espera el resultado sin bloquear — libera el hilo mientras tanto
6     await _context.Customers.AddAsync(customer);
7     await _context.SaveChangesAsync();
8 }
```

```
1 // Task<bool>: retorna un bool de forma asincrónica
2 public async Task<bool> ExistsByEmailAsync(string email)
3 {
```

```

4 // AnyAsync: verifica si existe algún registro que cumpla la condición
5 return await _context.Customers.AnyAsync(c => c.Email.Value == email);
6 }

```

Errores comunes

Error	Consecuencia
Olvidar <code>: base()</code> en la subclase	Si la clase base no tiene constructor sin parámetros, el compilador lo exige. Si lo omitís, el objeto base queda sin inicializar.
Constructor público en un VO	Si el constructor de un Value Object es <code>public</code> , cualquier código puede crear instancias sin pasar por <code>Create()</code> , saltando todas las validaciones. El estado inválido entra al dominio.
Método estático que accede a estado de instancia	Un método <code>static</code> no puede acceder a propiedades de instancia (<code>this.Nombre</code>). El compilador lo rechaza.
No usar <code>await</code> en un método async	Si se llama a un método async sin <code>await</code> , la operación se ejecuta en segundo plano sin esperar el resultado. Los datos pueden no estar disponibles cuando se los necesita.
Heredar de una clase <code>sealed</code>	El compilador rechaza la herencia de una clase <code>sealed</code> . Es intencional: <code>Customer</code> y los VOs son <code>sealed</code> para que nadie pueda alterar su comportamiento heredando de ellos.
No implementar todos los miembros de la interfaz	Si una clase declara implementar una interfaz pero omite algún miembro, el compilador lo rechaza con un error de compilación.

Qué NO hacer

Antipatrón 1 — Constructor público en un Value Object

```

1 // MAL: constructor público — cualquiera puede saltarse la validación
2 public sealed record Email : ValueObject
3 {
4     public string Value { get; }
5
6     // MAL: public — permite "new Email("no_es_email")" sin validar
7     public Email(string value) { Value = value; }
8 }

```

Antipatrón 2 — Llamar a método async sin `await`

```
1 // MAL: se llama al repositorio pero no se espera el resultado
2 public async Task Execute(CreateCustomerRequest request)
3 {
4     var customer = Customer.Create(...);
5
6     // MAL: falta await — el cliente puede no haberse guardado cuando se continúa
7     _repository.AddAsync(customer);
8
9     // BIEN: se usa await para garantizar que la persistencia se completó
10    await _repository.AddAsync(customer);
11 }
```

Antipatrón 3 — Heredar para reusar código sin relación semántica

```
1 // MAL: Order hereda de Customer solo para reutilizar Id y CreatedAt
2 // Un pedido NO ES UN cliente — la herencia viola LSP
3 public class Order : Customer { } // MAL
4
5 // BIEN: ambos heredan de BaseEntity — la relación semántica es correcta
6 public class Order : BaseEntity { } // BIEN
7 public class Customer : BaseEntity { } // BIEN
```